Sistema de Gestión de Biblioteca

Resolución de la consigna de diagrama de componentes

1. Lectura del enunciado y decisiones de diseño

La consigna pide modelar la arquitectura de un sistema que administra préstamos y devoluciones, consulta la disponibilidad de ejemplares, mantiene el catálogo, valida los datos de los libros con una biblioteca dinámica externa, persiste la información en una base de datos y genera un comprobante en PDF. Cada frase del enunciado se tradujo en un elemento del diagrama:

REQUERIMIENTO DEL ENUNCIADO	RESPUESTA EN EL MODELO
Administrar préstamos y devoluciones	**Servicio de Préstamos** («executable»): orquesta ambas operaciones y expone la interfaz `IPrestamos`.
El bibliotecario interactúa con el sistema	**Punto de Atención** («executable»), aplicación de escritorio asociada al actor; consume `IPrestamos`.
Verificar la disponibilidad de los ejemplares	**Gestión de Catálogo** responde `verificarDisponibilidad()` por `ICatalogo`; el dato se lee de la tabla **Ejemplar**.
Validar que los datos de los libros sean correctos	**ValidadorLibros.dll** («library»), biblioteca dinámica externa que provee `IValidacionLibros`.
Módulos independientes comunicados por interfaces	Cinco interfaces entre los módulos (`IPrestamos`, `ICatalogo`, `IValidacionLibros`, `IAccesoCatalogo` e `IRegistroPrestamos`), con conectores de ensamble bola–socket.
Libros, autores y ejemplares almacenados en una base de datos	**Repositorio Biblioteca** como único acceso a datos y paquete «database» con las tablas **Libro**, **Autor**, **Ejemplar** y **Prestamo**.
Comprobante en PDF al registrar un préstamo	Artefacto «document» **ComprobantePrestamo.pdf**, generado por el servicio (dependencia «genera»).
El servicio de préstamos debe ejecutarse en producción	**Servicio de Préstamos** se modela como «executable», desplegable en el servidor de producción.

4	5	4	6
COMPONENTES DE SOFTWARE	INTERFACES	TABLAS DE LA BASE	ESTEREOTIPOS UML

Frontera y supuestos. El sistema modelado es la biblioteca; el bibliotecario es el único actor humano y el usuario final solo recibe el comprobante. Se asume que la base de datos y la biblioteca dinámica ya existen en el entorno de producción. No se modelan el proveedor de libros ni el servicio de impresión.

2. Componentes y responsabilidades

ELEMENTO	ESTEREOTIPO	RESPONSABILIDAD	INTERFAZ PROVISTA	INTERFAZ REQUERIDA
Punto de Atención	«executable»	Interfaz de usuario del bibliotecario: registra préstamos y devoluciones, consulta disponibilidad y mantiene el catálogo.	—	`IPrestamos` · `ICatalogo`
Servicio de Préstamos	«executable»	Orquesta el préstamo y la devolución, controla plazos y dispara la generación del comprobante.	`IPrestamos`	`ICatalogo` · `IRegistroPrestamos`
Gestión de Catálogo	«component»	Mantiene libros, autores y ejemplares; valida los datos y responde por la disponibilidad.	`ICatalogo`	`IAccesoCatalogo` · `IValidacionLibros`
Repositorio Biblioteca	«component»	Único punto de acceso a la base; traduce las operaciones de los módulos a consultas sobre las tablas.	`IAccesoCatalogo` · `IRegistroPrestamos`	—
ValidadorLibros.dll	«library»	Biblioteca dinámica externa con las reglas de validación de los datos bibliográficos.	`IValidacionLibros`	—

3. Servicios e interfaces

Las interfaces se dibujan como interfaz provista (círculo o *lollipop*) del lado de quien ofrece el servicio y como interfaz requerida (*socket*) del lado de quien lo consume. Las operaciones concretas son:

INTERFAZ	PROVEEDOR	CONSUMIDOR	OPERACIONES PRINCIPALES
IPrestamos	Servicio de Préstamos	Punto de Atención	`registrarPrestamo()` · `registrarDevolucion()` · `consultarDisponibilidad()` · `consultarPrestamo()`
ICatalogo	Gestión de Catálogo	Punto de Atención · Servicio de Préstamos	`verificarDisponibilidad(idLibro)` · `validarLibro(datos)` · `obtenerEjemplares(idLibro)` · `registrarLibro()` · `actualizarEjemplar()`
IValidacionLibros	ValidadorLibros.dll	Gestión de Catálogo	`validarDatosLibro(datos)` → `ResultadoValidacion`
IAccesoCatalogo	Repositorio Biblioteca	Gestión de Catálogo	`buscarLibro()` · `guardarLibro()` · `actualizarEjemplar()`
IRegistroPrestamos	Repositorio Biblioteca	Servicio de Préstamos	`guardarPrestamo()` · `registrarDevolucion()` · `consultarPrestamos()`

> **Criterio de acoplamiento:** cada módulo depende solo de las operaciones que realmente usa. El Servicio de Préstamos no accede directamente a la base ni a la biblioteca de validación; lo hace a través de `ICatalogo` y de `IRegistroPrestamos`. Así, reemplazar la biblioteca dinámica o cambiar el motor de base de datos no obliga a modificar el resto del sistema.

4. Artefactos tecnológicos y su simbología UML

ARTEFACTO	ESTEREOTIPO	TIPO / FORMATO	ROL EN LA ARQUITECTURA
Punto de Atención	«executable»	Aplicación de escritorio (.exe)	Cliente instalado en el puesto del bibliotecario; no contiene reglas de negocio.
Servicio de Préstamos	«executable»	Servicio (.exe / .jar)	Se despliega y ejecuta en el entorno de producción, tal como exige el enunciado.
ValidadorLibros.dll	«library»	Biblioteca dinámica (.dll)	Reglas de validación externas; puede reemplazarse sin recompilar el sistema.
ComprobantePrestamo.pdf	«document»	Documento PDF	Se genera en cada préstamo y se entrega al usuario.
Biblioteca (base de datos)	«database»	Esquema relacional	Agrupa las tablas del sistema y separa el esquema físico del componente que lo accede.
Libro · Autor · Ejemplar · Prestamo	«table»	Tablas	Persistencia del catálogo, del stock disponible y de los préstamos con sus fechas de devolución.

5. Decisiones de diseño

- **Dos ejecutables y no uno:** el cliente (*Punto de Atención*) se separa del servicio de negocio (*Servicio de Préstamos*), porque el enunciado exige que el servicio de préstamos corra en producción mientras el bibliotecario lo usa desde su puesto. El cliente consume `IPrestamos` para el circuito de préstamos e `ICatalogo` para mantener el catálogo, que es un caso de uso independiente.
- **Una sola dependencia hacia la biblioteca dinámica:** la validación se concentra en *Gestión de Catálogo*; el servicio la invoca a través de `ICatalogo`. Se evita así duplicar el artefacto en el diagrama.
- **Se agregó la tabla Prestamo:** sin ella no puede registrarse una devolución ni conocerse qué ejemplar está prestado y desde cuándo.
- **Repositorio con dos interfaces:** `IAccesoCatalogo` e `IRegistroPrestamos` segregan las operaciones para que cada módulo dependa solo de las suyas.
- **Conectores de ensamble bola–socket:** muestran en un solo trazo quién provee y quién requiere la interfaz, en lugar de flechas sueltas con la etiqueta «usa».
- **Base modelada como «database»:** el paquete contiene las cuatro «table», de modo que el esquema físico queda separado del componente que lo consulta.

6. Verificación de la consigna

1. **Componentes identificados.** Cuatro componentes de software —dos de ellos ejecutables— con responsabilidades separadas.
2. **Servicios ofrecidos y requeridos.** Cada componente declara qué provee y qué necesita (tabla de la página 2).
3. **Interfaces de comunicación.** Cinco interfaces con sus operaciones: `IPrestamos`, `ICatalogo`, `IValidacionLibros`, `IAccesoCatalogo` e `IRegistroPrestamos`.
4. **Artefactos tecnológicos.** «executable» ×2, «library», «document», «database» y «table» ×4, con la simbología UML correspondiente.
5. **Diagrama UML.** En la página 4, con interfaces provistas y requeridas y las dependencias «genera» y «accede».

Entregables: diagrama en PDF/SVG/PNG y este documento de resolución.

«executable»
Artefacto que se ejecuta en un nodo de despliegue.

«library»
Biblioteca dinámica reutilizable, externa al código propio.

«component»
Módulo lógico reemplazable, agrupado por responsabilidad.

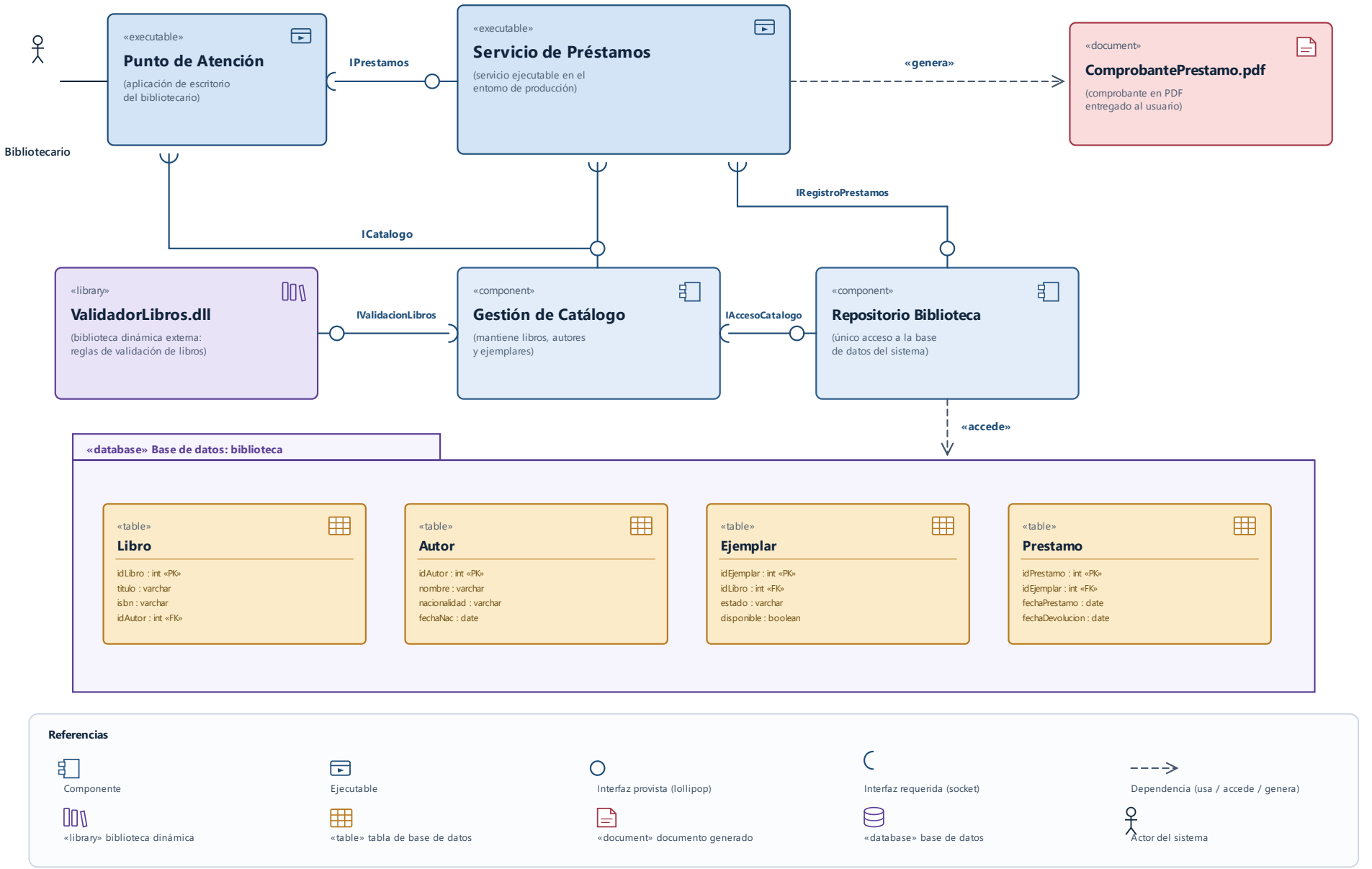
«database»
Esquema físico que contiene las tablas del sistema.

«table»
Tabla relacional persistente.

«document»
Archivo generado por el sistema en tiempo de ejecución.

7. Diagrama de componentes UML

Vista de alto nivel · notación UML 2.5



Bola: interfaz provista. **Socket:** interfaz requerida. **Línea punteada:** dependencia «genera» / «accede». **Azul:** componentes y ejecutables · **violeta:** biblioteca y base · **ámbar:** tablas · **rojo:** documento.